

PCP Theorem

[PCP Theorem is] the most important result in complexity theory since Cook's Theorem.

Ingo Wegener, 2005

S. Arora, C. Lund, R. Motwani, M. Sudan, and M. Szegedy. Proof Verification and the Hardness of Approximation Problems. J. ACM, 1998. FOCS 1992.



Irit Dinur. The PCP Theorem by Gap Amplification. J. ACM, 2007. STOC 2006.



The 1st proof is algebraic, the 2nd one is combinatorial and non-elementary.

Two ways to view the PCP Theorem:

- ▶ It is a result about locally testable proof systems.
- ▶ It is a result about hardness of approximation.

PCP = Probabilistically Checkable Proof

Synopsis

1. Approximation Algorithm
2. Two Views of PCP Theorem
3. Equivalence of the Two Views
4. Inapproximability
5. The Fourier Transform Technique
6. Efficient Conversion of NP Certificate to PCP Proof
7. Proof of PCP Theorem
8. Threshold Result by Håstad's 3-Bit PCP Theorem
9. Proof of Håstad's 3-Bit PCP Theorem
10. Historical Remark

Approximation Algorithm

Since the discovery of NP-completeness in 1972, researchers had been looking for approximate solutions to NP-hard **optimization** problems, with little success. The discovery of PCP Theorem in 1992 explains the difficulty.

Suppose $\rho : \mathbf{N} \rightarrow (0, 1)$. A ρ -approximation algorithm $\mathbb{A}$ for a maximum, respectively minimum optimization problem satisfies

$$\frac{\mathbb{A}(x)}{\text{Max}(x)} \geq \rho(|x|),$$

respectively

$$\frac{\text{Min}(x)}{\mathbb{A}(x)} \geq \rho(|x|)$$

for all x .

SubSet-Sum

Given m items of sizes $s_1, s_2, \dots, s_m$, and a positive integer C , find a subset of the items that maximizes the total sum of their sizes without exceeding the capacity C .

- ▶ There is a well-known dynamic programming algorithm.
- ▶ Using the algorithm and a parameter ϵ a $(1-\epsilon)$ -approximation algorithm can be designed that runs in $O\left(\left(\frac{1}{\epsilon} - 1\right) \cdot n^2\right)$ time.
- ▶ We say that SubSetSum has an **FPTAS**.

KnapSack

Let $U = \{u_1, u_2, \dots, u_m\}$ be the set of items to be packed in a knapsack of size C . For $1 \leq j \leq m$, let s_j and v_j be the size and value of the j -th item, respectively.

The objective is to fill the knapsack with items in U whose total size is at most C and such that their total value is maximum.

- ▶ There is a similar dynamic programming algorithm.
- ▶ Using the algorithm and a parameter ϵ one can design a $(1-\epsilon)$ -approximation algorithm of $O\left(\left(\frac{1}{\epsilon} - 1\right) \cdot n^{\frac{1}{\epsilon}}\right)$ time.
- ▶ We say that KnapSack has a **PTAS**.

Max-3SAT

For each 3CNF φ , the **value** of φ , denoted by $\text{val}(\varphi)$, is the **maximum fraction** of clauses that can be satisfied by an assignment to the variables of φ .

- ▶ φ is satisfiable if and only if $\text{val}(\varphi) = 1$.

Max-3SAT is the problem of finding the maximum $\text{val}(\varphi)$.

- ▶ A simple greedy algorithm for Max-3SAT is $\frac{1}{2}$ -approximate.
- ▶ We say that Max-3SAT is in **APX**.

By definition, $\text{FPTAS} \subseteq \text{PTAS} \subseteq \text{APX} \subseteq \text{OPT}$.

We will see that the inclusions are strict assuming $\mathbf{P} \neq \mathbf{NP}$.

Max-IS

$$\text{Min-VC} + \text{Max-IS} = m.$$

A $\frac{1}{2}$ -approximation algorithm for Min-VC.

1. Pick up a remaining edge and collect the two end nodes.
2. Remove all edges adjacent to the two nodes.
3. Goto Step 1 if there is at least one remaining edge.

-
- ▶ Is Min-VC in PTAS?
 - ▶ Is Max-IS in APX?

A breakthrough in the study of approximation algorithm was achieved in early 1990's.

[1991]. There is no $2^{\log^{1-\epsilon}(n)}$ -approximation algorithm for Max-IS unless $\text{SAT} \in \mathbf{SUBEXP}$.

[1992]. Max-IS is not in APX if $\mathbf{P} \neq \mathbf{NP}$.

1. U. Feige, S. Goldwasser, L. Lovász, S. Safra, and M. Szegedy. Interactive Proofs and the Hardness of Approximating Cliques. FOCS 1991. JACM, 1996.
2. S. Arora and S. Safra. Probabilistic Checking of Proofs: A New Characterization of NP. FOCS 1992. JACM, 1998.

Two Views of PCP Theorem

Surprisingly, **IP** = **PSPACE**. More surprisingly, **MIP** = **NEXP**.

The latter can be interpreted as saying that **nondeterminism** can be traded off for
randomness + **interaction**.

Interactive Proof Viewpoint

Suppose L is an NP problem and x is an input string.

1. Prover provides a **proof** π of **polynomial** length.
 2. Verifier uses at most **logarithmic** many **random** bits, and makes a **constant** number of **queries** on π .
-
- ▶ A query i is a **location** of **logarithmic** length. The answer to query i is $\pi(i)$.
 - ▶ Queries are written on a special address/oracle tape.
 - ▶ We assume that verifier is **nonadaptive** in that its selection of queries is based only on input and random string.

Probabilistically Checkable Proofs

Suppose L is a language and $q, r : \mathbf{N} \rightarrow \mathbf{N}$.

L has an $(r(n), q(n))$ -PCP verifier if a P-time verifier $\mathbb{V}$ exists satisfying the following.

- ▶ **Efficiency.** On input x and given access to any location of a proof π of length $\leq q(n)2^{r(n)}$, the verifier $\mathbb{V}$ uses at most $r(n)$ random bits and makes at most $q(n)$ nonadaptive queries to the proof π before it outputs '1' or '0'.
- ▶ **Completeness.** If $x \in L$, then $\exists \pi. \Pr[\mathbb{V}^\pi(x) = 1] = 1$.
- ▶ **Soundness.** If $x \notin L$, then $\forall \pi. \Pr[\mathbb{V}^\pi(x) = 1] \leq 1/2$.

$\mathbb{V}^\pi(x)$ denotes the random variable with x and π fixed.

Probabilistically Checkable Proofs

1. Proof length $\leq q(n)2^{r(n)}$.
 - ▶ At most $q(n)2^{r(n)}$ locations can be queried by verifier.
2. $L \in \mathbf{NTIME}(q(n)2^{O(r(n))})$.
 - ▶ An algorithm guesses a proof of length $q(n)2^{r(n)}$.
 - ▶ It executes deterministically $2^{r(n)}$ times the verifier's algorithm.
 - ▶ The total running time is bounded by $q(n)2^{O(r(n))}$.

Both random bits and query time are resources. An $(r(n), q(n))$ -PCP verifier has

- ▶ **randomness complexity** $r(n)$ and
- ▶ **query complexity** $q(n)$.

Sometimes one is concerned with **proof complexity** $q(n)2^{r(n)}$.

PCP Hierarchy

A language is in **PCP**($r(n), q(n)$) if it has a $(cr(n), dq(n))$ -PCP verifier for some c, d .

$$\mathbf{PCP}(r(n), q(n)) \subseteq \mathbf{NTIME}(q(n)2^{O(r(n))}).$$

- ▶ **PCP**(0, log) = **P**.
- ▶ **PCP**(0, poly) = **NP**.
- ▶ **PCP**(log, poly) = **NP**.

PCP Hierarchy

1. **PCP**(poly, poly) $\subseteq$ **NEXP**.
2. **PCP**(log, log) $\subseteq$ **NP**.
3. **PCP**(log, 1) $\subseteq$ **NP**.

In three influential papers in the history of PCP, it is proved that the above ' $\subseteq$ ' can be strengthened to '='.

The PCP Theorem

PCP Theorem. $\mathbf{NP} = \mathbf{PCP}(\log, 1)$.

Every NP-problem has specifically chosen certificates whose correctness can be verified probabilistically by checking only 3 bits.

Example

$\text{GNI} \in \mathbf{PCP}(\text{poly}, 1)$.

- ▶ Suppose both G_0 and G_1 have n vertices.
- ▶ Proofs of size 2^{n^2} are indexed by adjacent matrix representations.
 - ▶ If the location, a string of size n^2 , represents a graph isomorphic to G_i , it has value i .
- ▶ The verifier picks up $b \in \{0, 1\}$ at random, produces a random permutation of G_b , and queries the bit of the proof at the corresponding location.

Can we scale down PCP Theorem further?

Fact. If $\mathbf{NP} \subseteq \mathbf{PCP}(o(\log), o(\log))$, then $\mathbf{P} = \mathbf{NP}$.

Scale-Up PCP Theorem

Theorem. $\text{PCP}(\text{poly}, 1) = \text{NEXP}$.

Hardness of Approximation Viewpoint

For many NP-hard optimization problems, computing approximate solutions is no easier than computing the exact solutions.

The PCP Theorem, Hardness of Approximation

PCP Theorem. There exists $\rho < 1$ such that for every $L \in \mathbf{NP}$ there is a P-time computable function $f : L \rightarrow 3\text{SAT}$ such that

$$x \in L \Rightarrow \text{val}(f(x)) = 1,$$

$$x \notin L \Rightarrow \text{val}(f(x)) < \rho.$$

-
- Figure out the significance of the theorem by letting $L = 3\text{SAT}$.

The PCP Theorem, Hardness of Approximation

PCP Theorem cannot be proved using Cook-Levin reduction.

- ▶ $\text{val}(f(x))$ tends to 1 even if $x \notin L$.

“The intuitive reason is that computation is an inherently unstable, non-robust mathematical object, in the sense that it can be turned from non-accepting to accepting by changes that would be insignificant in any reasonable metric.”

Papadimitriou and Yannakakis, 1988

Corollary. There exists some $\rho < 1$ such that if there is a P-time ρ -approximation algorithm for Max-3SAT then **P** = **NP**.

- ▶ The ρ -approximation algorithm for Max-3SAT is NP-hard.

Equivalence of the Two Views

CSP, Constraint Satisfaction Problem

If q is a natural number, then a q CSP instance φ with n variables is a collection of constraints $\varphi_1, \dots, \varphi_m : \{0, 1\}^n \rightarrow \{0, 1\}$ such that for each $i \in [m]$ the function φ_i depends on q of its input locations.

We call q the arity of φ , and m the size of φ .

An assignment $\mathbf{u} \in \{0, 1\}^n$ satisfies a constraint φ_i if $\varphi_i(\mathbf{u}) = 1$. Let

$$\text{val}(\varphi) = \max_{\mathbf{u} \in \{0, 1\}^n} \left\{ \frac{\sum_{i=1}^m \varphi_i(\mathbf{u})}{m} \right\}.$$

We say that φ is satisfiable if $\text{val}(\varphi) = 1$.

q CSP is a generalization of 3SAT.

1. We assume that $n \leq qm$.
2. Since every φ_i can be described by a formula of size $q2^q$, and every variable can be coded up by $\log n$ bits, a q CSP instance can be described by $O(mq2^q \log n)$ bits.
3. The greedy algorithm for MAX-3SAT can be applied to MAX q CSP to produce an assignment satisfying $\geq \frac{\text{val}(\varphi)}{2^q} m$ constraints.

Gap CSP

Suppose $q \in \mathbf{N}$ and $\rho \leq 1$.

Let ρ -GAP q CSP be the promise problem of determining if a q CSP instance φ satisfies either (1) $\text{val}(\varphi) = 1$ or (2) $\text{val}(\varphi) < \rho$.

We say that ρ -GAP q CSP is NP-hard if for every NP-problem L some P-time computable function $f : L \rightarrow \rho$ -GAP q CSP exists such that

$$\begin{aligned}x \in L &\Rightarrow \text{val}(f(x)) = 1, \\x \notin L &\Rightarrow \text{val}(f(x)) < \rho.\end{aligned}$$

PCP Theorem. There exists some $\rho \in (0, 1)$ such that ρ -GAP3SAT is NP-hard.

PCP Theorem. There exist $q \in \mathbf{N}$ and $\rho \in (0, 1)$ such that ρ -GAP q CSP is NP-hard.

Equivalence Proof

PCP Theorem $\Rightarrow$ **PCP Theorem**.

This is essentially the Cook-Levin reduction.

1. Suppose $\mathbf{NP} \subseteq \mathbf{PCP}(\log, 1)$. Then 3SAT has a PCP verifier $\mathbb{V}$ that makes q queries using $c \log n$ random bits.
2. Given input x with $|x| = n$ and random string $r \in \{0, 1\}^{c \log n}$, $\mathbb{V}(x, r)$ is a Boolean function of type $\{0, 1\}^q \rightarrow \{0, 1\}$.
3. $\varphi = \{\mathbb{V}(x, r)\}_{r \in \{0, 1\}^{c \log n}}$ is a P-size q CSP instance.
 - ▶ By completeness, $x \in 3\text{SAT} \Rightarrow \text{val}(\varphi) = 1$.
 - ▶ By soundness, $x \notin 3\text{SAT} \Rightarrow \text{val}(\varphi) \leq 1/2$.
4. The map from 3SAT to $\frac{1}{2}$ -GAP q CSP is P-time computable.
 - ▶ $\mathbb{V}$ runs in P-time.

Equivalence Proof

PCP Theorem $\Leftarrow$ **PCP Theorem**.

Suppose $L \in \mathbf{NP}$ and ρ -GAP q CSP is NP-hard for some $q \in \mathbf{N}$, $\rho < 1$. By assumption there is some P-time reduction $f : L \rightarrow \rho$ -GAP q CSP.

1. The verifier for L works as follows:
 - ▶ On input x , compute the q CSP instance $f(x) = \{\varphi_i\}_{i \in [m]}$.
 - ▶ Given a proof π , which is an assignment to the variables, it randomly chooses $i \in [m]$ and checks if φ_i is satisfied by reading the relevant q bits of the proof.
2. If $x \in L$, the verifier always accepts; otherwise it accepts with probability $< \rho$.

Equivalence Proof

PCP Theorem $\Leftrightarrow$ PCP Theorem.

This is very much like the equivalence between SAT and 3SAT.

1. Let $\epsilon > 0$ and $q \in \mathbf{N}$ be such that $(1-\epsilon)$ -GAP q CSP is NP-hard.
2. Let $\varphi = \{\varphi_i\}_{i=1}^m$ be a q CSP instance with n variables.
3. Each φ_i is the conjunction of at most 2^q clauses, each being the disjunction of at most q literals.
4. If all assignments fail at least an ϵ fragment of the constraints of φ , then all assignments fail at least a $\frac{\epsilon}{q2^q}$ fragment of the clauses of the 3SAT instance.

Proof View	Inapproximability View
PCP verifier V PCP proof π proof length $ \pi $ number of queries q number of random bits r soundness parameter ϵ	CSP instance φ assignment to variables $\mathbf{u}$ number of variables n arity of constraints q logarithm of number of constraints $\log m$ maximum fraction of violated constraints of no instance
NP $\subseteq$ PCP (log, 1)	ρ -GAP q CSP is NP-hard

The equivalence of the proof view and the inapproximability view is essentially due to the Cook-Levin Theorem for PTM.

Inapproximability

Min-VC and Max-IS are inherently different from the perspective of approximation.

- ▶ $\text{Min-VC} + \text{Max-IS} = n$.
- ▶ ρ -approximation algorithm of Max-IS $\Rightarrow \frac{n-IS}{n-\rho IS}$ -approximation algorithm of Min-VC.

Lemma. There is a P-time computable function f from 3CNF formulas to graphs that maps a formula φ to an n -vertex graph $f(\varphi)$ whose independent set is of size $\text{val}(\varphi)\frac{n}{7}$.

The standard Karp reduction from 3SAT to Max-IS is as follows:

- ▶ Each clause is translated to a clique of 7 nodes, each node represents a (partial) assignment that validates the clause.
- ▶ Two nodes from two different cliques are connected if and only if they conflict.

A 3CNF formula φ consisting of m clauses is translated to a graph with $7m$ nodes, and an assignment satisfying l clauses of φ if and only if the graph has an independent set of size l .

Theorem. The following statements are valid.

1. $\exists \rho' < 1$. ρ' -approximation to Min-VC is NP-hard, and
2. $\forall \rho < 1$. ρ -approximation to Max-IS is NP-hard.

[$\exists \rho$. ρ -approximation to Max-IS is NP-hard.]* By PCP Theorem, ρ -approximation to Max-3SAT is NP-hard for some ρ . So by Lemma ρ -approximation to Max-IS is NP-hard.

1. Referring to the map of Lemma, the minimum vertex cover has size $n - \text{val}(\varphi) \frac{n}{7}$. Let $\rho' = \frac{6}{7-\rho}$. Suppose Min-VC had a ρ' -approximation algorithm.

- ▶ If $\text{val}(\varphi) = 1$, it would produce a vertex cover of size $\leq \frac{1}{\rho'}(n - \frac{n}{7}) = n - \rho \frac{n}{7}$.
- ▶ If $\text{val}(\varphi) < \rho$, the minimum vertex cover has size $> n - \rho \frac{n}{7}$. The algorithm must return a vertex cover of size $> n - \rho \frac{n}{7}$.

The first proposition is established. The second will be proved by making use of [−]*.

2. Assume that Max-IS were ρ_0 -approximate. Let k satisfy $\rho_0\binom{IS}{k} > \binom{\rho IS}{k}$.
 1. Suppose G is the input graph. Construct G^k as follows:
 - ▶ The vertices are k -size subsets of V_G ;
 - ▶ Two vertices S_1, S_2 are **dis**connected if $S_1 \cup S_2$ is an independent set of G .
 2. Apply the ρ_0 -approximation algorithm to G^k , and derive an independent set of G from the output of the algorithm.
-
- ▶ The largest independent set of G^k is of size $\binom{IS}{k}$, where IS is the maximum independent set of G .
 - ▶ The output is an independent set of size $\geq \rho_0\binom{IS}{k} > \binom{\rho IS}{k}$.
 - ▶ An independent set of size ρIS can be derived. This is a contradiction.

The Fourier Transform Technique

A Boolean function $f : \{0, 1\}^n \rightarrow \{0, 1\}$ is a **linear function** if

$$f(\mathbf{x} + \mathbf{y}) = f(\mathbf{x}) + f(\mathbf{y}),$$

where “+” is the exclusive-or operator.

Fourier Transform over $GF(2)^n$

Boolean functions can be studied using Fourier transform over $GF(2)^n$.

We shall use $\{+1, -1\}$ instead of $\{0, 1\}$ in this section.

- ▶ $0 \leftrightarrow (-1)^0 = 1$ and $1 \leftrightarrow (-1)^1 = -1$.
- ▶ $\{0, 1\}^n$ is turned into $\{\pm 1\}^n$.
- ▶ "exclusive-or" is turned into "multiplication".

Fourier Transform over $GF(2)^n$

The 2^n -dimensional Hilbert space $\mathbf{R}^{\{\pm 1\}^n}$ is defined as follows: For $f, g \in \mathbf{R}^{\{\pm 1\}^n}$,

1. $(f + g)(\mathbf{x}) = f(\mathbf{x}) + g(\mathbf{x})$,
2. $(cf)(\mathbf{x}) = cf(\mathbf{x})$, and
3. expectation inner product: $\langle f, g \rangle = \mathbb{E}_{\mathbf{x} \in \{\pm 1\}^n} [f(\mathbf{x})g(\mathbf{x})]$.

Standard orthogonal basis: $\{\mathbf{e}_{\mathbf{x}}\}_{\mathbf{x} \in \{\pm 1\}^n}$.

Fourier Transform over $GF(2)^n$

Fourier Basis: $\{\chi_\alpha\}_{\alpha \subseteq [n]}$, where $\chi_\alpha(\mathbf{x}) = \prod_{i \in \alpha} x_i$.

1. $\chi_\emptyset = \mathbf{1}$.
2. We will see that Fourier basis functions are the same as the linear functions.
3. Inner product $\langle \chi_\alpha, \chi_\beta \rangle = 1$ if $\alpha = \beta$ and $= 0$ otherwise.

Fourier basis is orthonormal.

- ▶ $\langle \chi_\alpha, \chi_\alpha \rangle = E_{\mathbf{x} \in \{\pm 1\}^n}[\chi_\alpha(\mathbf{x})\chi_\alpha(\mathbf{x})] = E_{\mathbf{x} \in \{\pm 1\}^n}[\chi_\alpha(\mathbf{x} \cdot \mathbf{x})] = E_{\mathbf{x} \in \{\pm 1\}^n}[1] = 1$.
- ▶ $\langle \chi_\alpha, \chi_\beta \rangle = E_{\mathbf{x} \in \{\pm 1\}^n}[\chi_\alpha(\mathbf{x})\chi_\beta(\mathbf{x})] = 0$ if $\alpha \neq \beta$.

The notation $\mathbf{y} \cdot \mathbf{z}$ stands for the dot product of $\mathbf{y}$ and $\mathbf{z}$.

Fourier Transform over $GF(2)^n$

$f = \sum_{\alpha \subseteq [n]} \hat{f}_\alpha \chi_\alpha$ for every $f \in \mathbf{R}^{\{\pm 1\}^n}$, where $\hat{f}_\alpha$ is the α th Fourier coefficient of f .

Lemma. (i) $\langle f, g \rangle = \sum_{\alpha \subseteq [n]} \hat{f}_\alpha \hat{g}_\alpha$. (ii) (Parseval's Identity) $\langle f, f \rangle = \sum_{\alpha \subseteq [n]} \hat{f}_\alpha^2$.

Proof.

$$\langle f, g \rangle = \langle \sum_{\alpha \subseteq [n]} \hat{f}_\alpha \chi_\alpha, \sum_{\beta \subseteq [n]} \hat{g}_\beta \chi_\beta \rangle = \sum_{\alpha, \beta \subseteq [n]} \hat{f}_\alpha \hat{g}_\beta \langle \chi_\alpha, \chi_\beta \rangle = \sum_{\alpha \subseteq [n]} \hat{f}_\alpha \hat{g}_\alpha. \quad \square$$

Fourier Transform over $GF(2)^n$

Example.

1. Majority function of 3 variables $= \frac{1}{2}u_1 + \frac{1}{2}u_2 + \frac{1}{2}u_3 - \frac{1}{2}u_1u_2u_3$.
2. Projection function $\lambda_{x_1 \dots x_n}.x_i$. Here $\hat{f}_\alpha$ is 1 if $\alpha = \{i\}$ and is 0 if $\alpha \neq \{i\}$.

Fourier Transform over $GF(2)^n$

Theorem. Suppose $f : \{\pm 1\}^n \rightarrow \{\pm 1\}$ satisfies $\Pr_{\mathbf{x}, \mathbf{y}}[f(\mathbf{x} \cdot \mathbf{y}) = f(\mathbf{x})f(\mathbf{y})] \geq \frac{1}{2} + \epsilon$. Then there is some $\alpha \subseteq [n]$ such that $\hat{f}_\alpha \geq 2\epsilon$.

The assumption is equivalent to $E_{\mathbf{x}, \mathbf{y}}[f(\mathbf{x} \cdot \mathbf{y})f(\mathbf{x})f(\mathbf{y})] \geq \frac{1}{2} + \epsilon - (\frac{1}{2} - \epsilon) = 2\epsilon$. Now

$$\begin{aligned} 2\epsilon &\leq E_{\mathbf{x}, \mathbf{y}}[f(\mathbf{x} \cdot \mathbf{y})f(\mathbf{x})f(\mathbf{y})] &= E_{\mathbf{x}, \mathbf{y}}[(\sum_{\alpha} \hat{f}_\alpha \chi_\alpha(\mathbf{x} \cdot \mathbf{y}))(\sum_{\beta} \hat{f}_\beta \chi_\beta(\mathbf{x}))(\sum_{\gamma} \hat{f}_\gamma \chi_\gamma(\mathbf{y}))] \\ &= E_{\mathbf{x}, \mathbf{y}}[\sum_{\alpha, \beta, \gamma} \hat{f}_\alpha \hat{f}_\beta \hat{f}_\gamma \chi_\alpha(\mathbf{x}) \chi_\alpha(\mathbf{y}) \chi_\beta(\mathbf{x}) \chi_\gamma(\mathbf{y})] \\ &= \sum_{\alpha, \beta, \gamma} \hat{f}_\alpha \hat{f}_\beta \hat{f}_\gamma E_{\mathbf{x}, \mathbf{y}}[\chi_\alpha(\mathbf{x}) \chi_\alpha(\mathbf{y}) \chi_\beta(\mathbf{x}) \chi_\gamma(\mathbf{y})] \\ &= \sum_{\alpha, \beta, \gamma} \hat{f}_\alpha \hat{f}_\beta \hat{f}_\gamma E_{\mathbf{x}}[\chi_\alpha(\mathbf{x}) \chi_\beta(\mathbf{x})] E_{\mathbf{y}}[\chi_\alpha(\mathbf{y}) \chi_\gamma(\mathbf{y})] \\ &= \sum_{\alpha} \hat{f}_\alpha^3 \leq (\max_{\alpha} \hat{f}_\alpha) \sum_{\alpha} \hat{f}_\alpha^2 = \max_{\alpha} \hat{f}_\alpha. \end{aligned}$$

The last equality is due to the fact that f is Boolean.

Fourier Transform over $GF(2)^n$

For Boolean functions $f, g : \{\pm 1\}^n \rightarrow \{\pm 1\}$, the inner product $\langle f, g \rangle$ is

$$\frac{|\{\mathbf{x} \mid f(\mathbf{x}) = g(\mathbf{x})\}|}{2^n} - \frac{|\{\mathbf{x} \mid f(\mathbf{x}) \neq g(\mathbf{x})\}|}{2^n}.$$

Referring to the Theorem, let $\hat{f}_\alpha \geq 2\epsilon$ be the largest Fourier coefficient of f . Then

$$\langle f, \chi_\alpha \rangle \geq 2\epsilon.$$

In other words f coincides with the basis function χ_α on $\geq \frac{1}{2} + \epsilon$ fraction of inputs.

Efficient Conversion of NP Certificate to PCP Proof

Proofs of PCP Theorems involve some interesting ways of encoding NP-certificates and the associated methods of checking if a string is a valid encoding.

One idea is to amplify any error that appears in an NP-certificate.

We shall showcase how it works by looking at a problem to which the amplification power of Walsh-Hadamard Code can be exploited.

Theorem. $\text{NP} \subseteq \text{PCP}(\text{poly}(n), 1)$.

Walsh-Hadamard Code

The **Walsh-Hadamard function** $\text{WH} : \{0, 1\}^n \rightarrow \{0, 1\}^{2^n}$ encodes a string of length n by a function in n variables over $\text{GF}(2)$:

$$\text{WH}(\mathbf{u}) : \mathbf{x} \mapsto \mathbf{u} \odot \mathbf{x},$$

where $\mathbf{u} \odot \mathbf{x} = \sum_{i=1}^n u_i x_i \pmod{2}$.

Random Subsum Principle.

- ▶ If $\mathbf{u} \neq \mathbf{v}$ then for exactly half the choices of $\mathbf{x}$, $\mathbf{u} \odot \mathbf{x} \neq \mathbf{v} \odot \mathbf{x}$.

Walsh-Hadamard Codeword

We say that f is a **Walsh-Hadamard codeword** if $f = \text{WH}(\mathbf{u})$ for some $\mathbf{u} \in \{0, 1\}^n$.

1. Walsh-Hadamard codewords are precisely the linear functions. This is because a linear function f is the same as $\text{WH}(\mathbf{f})$, where

$$\mathbf{f} = \begin{pmatrix} f(\mathbf{e}_1) \\ f(\mathbf{e}_2) \\ \vdots \\ f(\mathbf{e}_n) \end{pmatrix}.$$

2. Walsh-Hadamard codewords are essentially the Fourier basis functions.

Let $\rho \in [0, 1]$. The functions $f, g : \{0, 1\}^n \rightarrow \{0, 1\}$ are ρ -close if

$$\Pr_{\mathbf{x} \in_{\mathbb{R}} \{0,1\}^n} [f(\mathbf{x}) = g(\mathbf{x})] \geq \rho.$$

Theorem. Let $f : \{0, 1\}^n \rightarrow \{0, 1\}$ be such that for some $\rho > \frac{1}{2}$,

$$\Pr_{\mathbf{x}, \mathbf{y} \in_{\mathbb{R}} \{0,1\}^n} [f(\mathbf{x} + \mathbf{y}) = f(\mathbf{x}) + f(\mathbf{y})] \geq \rho.$$

Then f is ρ -close to a linear function.

Proof.

Let $\rho = \frac{1}{2} + \epsilon$. This follows immediately from the theorem of the previous section. $\square$

Local Testing of Walsh-Hadamard Codeword

A **local test** of f checks if f is a Walsh-Hadamard codeword by making a **constant** number of queries.

- ▶ It accepts every linear function, and
- ▶ it rejects every function that is far from being linear with high probability.

For $\delta \in (0, 1/2)$ a **$(1 - \delta)$ -linearity test** rejects with probability $\geq \frac{1}{2}$ any function not $(1 - \delta)$ -close to a linear function by testing

$$f(\mathbf{x} + \mathbf{y}) = f(\mathbf{x}) + f(\mathbf{y})$$

randomly for $\frac{1}{\delta}$ times.

The error probability is $\leq (1 - \delta)^{\frac{1}{\delta}} \approx \frac{1}{e} < \frac{1}{2}$.

Local Decoding

Suppose $\delta < \frac{1}{4}$ and f is $(1 - \delta)$ -close to some linear function $\tilde{f}$.

Given $\mathbf{x}$ one can learn $\tilde{f}(\mathbf{x})$ by making only two queries to f .

1. Choose $\mathbf{x}' \in_{\mathcal{R}} \{0, 1\}^n$;
 2. Set $\mathbf{x}'' = \mathbf{x} + \mathbf{x}'$;
 3. Output $f(\mathbf{x}') + f(\mathbf{x}'')$.
-

By union bound $\tilde{f}(\mathbf{x}) = \tilde{f}(\mathbf{x}') + \tilde{f}(\mathbf{x}'') = f(\mathbf{x}') + f(\mathbf{x}'')$ holds with probability $\geq 1 - 2\delta$.

Quadratic Equation in GF(2)

Suppose $\mathbf{A}$ is an $m \times n^2$ matrix and $\mathbf{b}$ is an m -dimensional vector with values in GF(2). Let $(\mathbf{A}, \mathbf{b}) \in \text{QUADEQ}$ if there is an n -dimensional vector $\mathbf{u}$ such that

$$\mathbf{A}(\mathbf{u} \otimes \mathbf{u}) = \mathbf{b},$$

where $\mathbf{u} \otimes \mathbf{u}$ is the tensor product of $\mathbf{u}$.

$$\mathbf{u} \otimes \mathbf{u} = (u_1 u_1, \dots, u_1 u_n, \dots, u_n u_1, \dots, u_n u_n)^\dagger.$$

Quadratic Equation in GF(2)

An instance of QUADEQ over u_1, u_2, u_3, u_4, u_5 :

$$u_1 u_2 + u_3 u_4 + u_1 u_5 = 1$$

$$u_1 u_1 + u_2 u_3 + u_1 u_4 = 0$$

A satisfying assignment is $(0, 0, 1, 1, 0)$.

QUADEQ is NP-Complete

$\text{CKT-SAT} \leq_K \text{QUADEQ}$.

- ▶ The inputs and the outputs are turned into variables.
- ▶ Boolean equality $x \vee y = z$ relating the inputs to the output is turned into algebraic equality $(1 - x)(1 - y) = 1 - z$ in $\text{GF}(2)$, which is equivalent to $xx + yy + xy + zz = 0$.
- ▶ $\neg x = z$ is turned into $xx + zz = 1$.
- ▶ $x \wedge y = z$ is turned into $xy + zz = 0$.

From NP Certificate to PCP Proof

A certificate for $(\mathbf{A}, \mathbf{b})$ is an n -dimensional vector $\mathbf{u}$ witnessing $(\mathbf{A}, \mathbf{b}) \in \text{QUADEQ}$.

- ▶ To check if $\mathbf{u}$ is a solution, one reads the n bits of $\mathbf{u}$ and checks the m equations.

We convert an NP-certificate $\mathbf{u}$ to the PCP-proof $\text{WH}(\mathbf{u})\text{WH}(\mathbf{u} \otimes \mathbf{u})$.

- ▶ The proof is a string of length $2^n + 2^{n^2}$.
- ▶ Using the proof it is straightforward to verify probabilistically if $(\mathbf{A}, \mathbf{b}) \in \text{QUADEQ}$.

Verifier for QUADEQ

Step 1. Verify that f, g are linear functions.

1. Perform a 0.999-linearity test on f, g .

If successful we may assume that $f(\mathbf{r}) = \mathbf{u} \odot \mathbf{r}$ and $g(\mathbf{z}) = \mathbf{w} \odot \mathbf{z}$.

Verifier for QUADEQ

Step 2. Verify that g encodes $(\mathbf{u} \otimes \mathbf{u}) \odot _.$

1. Get independent random $\mathbf{r}, \mathbf{r}'$.
2. Reject if $f(\mathbf{r})f(\mathbf{r}') \neq g(\mathbf{r} \otimes \mathbf{r}')$.
3. Repeat the test 10 times.

-
- ▶ In a correct proof $f(\mathbf{r})f(\mathbf{r}') = (\sum_i u_i r_i)(\sum_j u_j r'_j) = \sum_{i,j} u_i u_j r_i r'_j = g(\mathbf{r} \otimes \mathbf{r}')$.
 - ▶ Assume $\mathbf{w} \neq \mathbf{u} \otimes \mathbf{u}$. Let matrices $\mathbf{W}$ and $\mathbf{U}$ be $\mathbf{w}$ and respectively $\mathbf{u} \otimes \mathbf{u}$. One has
 - ▶ $g(\mathbf{r} \otimes \mathbf{r}') = \mathbf{w} \odot (\mathbf{r} \otimes \mathbf{r}') = \sum_{i,j} w_{ij} r_i r'_j = \mathbf{r} \mathbf{W} \mathbf{r}'$, and
 - ▶ $f(\mathbf{r})f(\mathbf{r}') = (\mathbf{u} \otimes \mathbf{r})(\mathbf{u} \otimes \mathbf{r}') = (\sum_i u_i r_i)(\sum_j u_j r'_j) = \mathbf{r} \mathbf{U} \mathbf{r}'$.
- $\mathbf{r} \mathbf{W}$, $\mathbf{r} \mathbf{U}$ differ for at least $\frac{1}{2}$ of $\mathbf{r}$'s; and $\mathbf{r} \mathbf{W} \mathbf{r}'$, $\mathbf{r} \mathbf{U} \mathbf{r}'$ differ for at least $\frac{1}{4}$ of $(\mathbf{r}, \mathbf{r}')$'s.
- ▶ The overall probability of rejection is at least $1 - (\frac{3}{4})^{10} > 0.9$.

Verifier for QUADEQ

Step 3. Verify that g encodes a solution.

1. Take a random subset S of $[m]$.
2. Reject if $g(\sum_{k \in S} A_{k,-}) \neq \sum_{k \in S} b_k$.

-
- ▶ There is enough time to check $\mathbf{A}(\mathbf{u} \otimes \mathbf{u}) = \mathbf{b}$ by checking, for every $k \in [m]$, the equality $g(A_{k,-}) = b_k$.
 - ▶ However since m is part of the input, the number of queries, which must be a constant, should **not** depend on m .
 - ▶ If $\{k \in [m] \mid g(A_{k,-}) \neq b_k\} \neq \emptyset$, then

$$\Pr_{S \subseteq_R [m]}[|S \cap \{k \in [m] \mid g(A_{k,-}) \neq b_k\}| \text{ is odd}] = \frac{1}{2}.$$

Note that $g(\sum_{k \in S} A_{k,-}) = \sum_{k \in S} g(A_{k,-})$ by linearity.

Suppose the PCP verifier for QUADEQ makes q_0 queries.

It follows from the completeness of QUADEQ that **all** NP problems have PCP verifiers that toss coins for a polynomial number of time and make precisely q_0 queries.

Proof of PCP Theorem

CSP with Nonbinary Alphabet

$q\text{CSP}_W$ is analogous to $q\text{CSP}$ except that the alphabet is $[W]$ instead of $\{0, 1\}$.

The constraints are functions of type $[W]^q \rightarrow \{0, 1\}$.

For $\rho \in (0, 1)$, we define the promise problem $\rho\text{-GAP}q\text{CSP}_W$ analogous to $\rho\text{-GAP}q\text{CSP}$.

3COL is a case of 2CSP_3 .

PCP Theorem states that ρ -GAP q CSP is NP-hard for some q, ρ .

The proof we shall describe is based on the following:

1. If φ of m constraints is unsatisfied, then $\text{val}(\varphi) \leq 1 - \frac{1}{m}$.
 2. There is a construction that increases the gap.
-

The idea is to start with an NP-problem, then apply Step 2 for $\log m$ times.

Let f be a function mapping CSP instances to CSP instances.

It is a **CL-reduction** (complete linear-blowup reduction) if it is P-time computable and the following are valid for every CSP instance φ .

- ▶ **Completeness**. If φ is satisfiable then $f(\varphi)$ is satisfiable.
- ▶ **Linear Blowup**. If φ has m constraints, $f(\varphi)$ has no more than Cm constraints over a new alphabet W .
 - ▶ C, W depend on neither the number of constraints nor the number of variables of φ .

Main Lemma. There exist constants $q_0 \geq 3$, $\epsilon_0 > 0$ and CL-reduction f such that for every q_0 CSP instance φ and every $\epsilon < \epsilon_0$, $f(\varphi)$ is a q_0 CSP instance satisfying

$$\text{val}(\varphi) \leq 1 - \epsilon \Rightarrow \text{val}(f(\varphi)) \leq 1 - 2\epsilon.$$

q_0 CSP Instance	Arity	Alphabet	Constraint	Gap
φ	q_0	binary	m	$1 - \epsilon$
$\Downarrow$	$\Downarrow$	$\Downarrow$	$\Downarrow$	$\Downarrow$
$f(\varphi)$	q_0	binary	Cm	$1 - 2\epsilon$

In the following proof of the PCP Theorem, q_0 , ϵ_0 , ℓ , W , W' , d , t are all constants. They depend neither on the number n of variables nor on the number m of constraints.

Proof of PCP Theorem

Let $q_0 \geq 3$ and $\epsilon_0 > 0$ be given by the Main Lemma. A CL-reduction from $q_0\text{CSP}$ to $(1-2\epsilon_0)\text{-GAP}q_0\text{CSP}$ is obtained as follows:

1. $q_0\text{CSP}$ is NP-hard.
2. For a $q_0\text{CSP}$ instance φ with m constraints, apply Main Lemma for $\log m$ times to amplify the gap. We get an instance ψ .
3. If φ is satisfiable, then ψ is satisfiable. Otherwise according to Main Lemma

$$\text{val}(\psi) \leq 1 - 2^{\log m} \cdot \frac{1}{m} \leq 1 - 2\epsilon_0.$$

4. $|\psi| \leq C^{\log m} m = \text{poly}(|\varphi|)$. Conclude that $(1-2\epsilon_0)\text{-GAP}q_0\text{CSP}$ is NP-hard.

C depends on two constants, q_0 and 2 (the size of alphabet).

Main Lemma is proved in three steps.

1. Prove that every q CSP instance can be turned into a “nice” q CSP _{W} instance.
2. **Gap Amplification**. Construct a CL-reduction f that increases both the gap and the alphabet size of a “nice” q CSP instance. [Dinur’s proof]
3. **Alphabet Reduction**. Construct a CL-reduction g that decreases the alphabet size to 2 with a modest reduction in the gap. [Proof of Arora et al.]

Gap Amplification. For **all** numbers ℓ, q , there are number $W, \epsilon_0 \in (0, 1)$ and a CL-reduction $g_{\ell, q}$ such that for every q CSP instance φ , $\psi = g_{\ell, q}(\varphi)$ is a 2CSP_W instance that satisfies the following for all $\epsilon < \epsilon_0$.

$$\text{val}(\varphi) \leq 1 - \epsilon \Rightarrow \text{val}(\psi) \leq 1 - \ell\epsilon.$$

Alphabet Reduction. There **exist** a constant q_0 and a CL-reduction h such that for every 2CSP_W instance φ , $\psi = h(\varphi)$ is a q_0 CSP instance satisfying

$$\text{val}(\varphi) \leq 1 - \epsilon \Rightarrow \text{val}(\psi) \leq 1 - \epsilon/3.$$

CSP Instance	Arity	Alphabet	Constraint	Gap
φ	q_0	binary	m	$1 - \epsilon$
$\Downarrow$ $f(\varphi)$	$\Downarrow$ 2	$\Downarrow$ nonbinary	$\Downarrow$ $C'm$	$\Downarrow$ $1 - 6\epsilon$
$\Downarrow$ $g(f(\varphi))$	$\Downarrow$ q_0	$\Downarrow$ binary	$\Downarrow$ $C''C'm$	$\Downarrow$ $1 - 2\epsilon$

Dinur makes use of expander graphs to construct new constraints.

Let φ be a 2CSP_W instance with n variables. The **constraint graph** G_φ of φ is defined as follows:

1. the vertex set is $[n]$, and
2. (i, j) is an edge if there is a constraint on the variables u_i, u_j . Parallel edges and self-loops are allowed.

A 2CSP_W instance φ is nice if the following are valid:

1. There is a constant d such that G_φ is a $(d, 0.9)$ -expander.
2. At every vertex half of the adjacent edges are self loops.

A nice CSP instance looks like an expander. In a nice CSP a $t + 1$ step random walk is very much like a t step random walk.

Lemma. Let G be a d -regular n -vertex graph, S be a vertex subset and $T = \bar{S}$. Then

$$|E(S, T)| \geq (1 - \lambda_G) \frac{d|S||T|}{|S| + |T|}. \quad (1)$$

The vector $\mathbf{x}$ defined below satisfies $\|\mathbf{x}\|_2^2 = |S||T|(|S| + |T|)$ and $\mathbf{x} \perp \mathbf{1}$.

$$\mathbf{x}_i = \begin{cases} +|T|, & i \in S, \\ -|S|, & i \in T. \end{cases}$$

Let $Z = \sum_{i,j} A_{i,j}(\mathbf{x}_i - \mathbf{x}_j)^2$. By definition $Z = \frac{2}{d}|E(S, T)|(|S| + |T|)^2$. On the other hand

$$Z = \sum_{i,j} A_{i,j} \mathbf{x}_i^2 - 2 \sum_{i,j} A_{i,j} \mathbf{x}_i \mathbf{x}_j + \sum_{i,j} A_{i,j} \mathbf{x}_j^2 = 2\|\mathbf{x}\|_2^2 - 2\langle \mathbf{x}, A\mathbf{x} \rangle.$$

Since $\mathbf{x} \perp \mathbf{1}$, $\langle \mathbf{x}, A\mathbf{x} \rangle \leq \lambda_G \|\mathbf{x}\|_2^2$ (cf. Rayleigh quotient).

Let $G = (V, E)$ be an expander and $S \subseteq V$ with $|S| \leq |V|/2$. The following holds.

$$\Pr_{(u,v) \in E}[u \in S, v \in S] \leq \frac{|S|}{|V|} \left(\frac{1}{2} + \frac{\lambda_G}{2} \right). \quad (2)$$

Observe that $|S|/|V| = \Pr_{(u,v) \in E}[u \in S, v \in S] + \Pr_{(u,v) \in E}[u \in S, v \in \bar{S}]$. And by (1), one has

$$\Pr_{(u,v) \in E}[u \in S, v \in \bar{S}] = E(S, \bar{S})/d|V| \geq \frac{|S|}{|V|} \cdot \frac{1}{2} \cdot (1 - \lambda_G).$$

We are done by substituting $|S|/|V| - \Pr_{(u,v) \in E}[u \in S, v \in S]$ for $\Pr_{(u,v) \in E}[u \in S, v \in \bar{S}]$.

$$\Pr_{(u,v) \in E^\ell}[u \in S, v \in S] \leq \frac{|S|}{|V|} \left(\frac{1}{2} + \frac{\lambda_G^\ell}{2} \right). \quad (3)$$

Step 1: Reduction to Nice Instance

The reduction consists of three steps.

$$\begin{aligned} q_0 \text{CSP instance} &\stackrel{\text{Step1.1}}{\Rightarrow} 2\text{CSP}_{2^{q_0}} \text{ instance} \\ &\stackrel{\text{Step1.2}}{\Rightarrow} 2\text{CSP}_{2^{q_0}} \text{ instance with regular constraint graph} \\ &\stackrel{\text{Step1.3}}{\Rightarrow} \text{nice } 2\text{CSP}_{2^{q_0}} \text{ instance.} \end{aligned}$$

In all the three steps the fraction of violated constraints decreases.

Step 1: Reduction to Nice Instance

Step 1.1. There exists a CL-reduction that maps a q_0 CSP instance φ to a $2\text{CSP}_{2^{q_0}}$ instance ψ such that

$$\text{val}(\varphi) \leq 1 - \epsilon \Rightarrow \text{val}(\psi) \leq 1 - \frac{\epsilon}{q_0}.$$

Suppose φ has variables $x_1, \dots, x_n$ and m constraints.

- ▶ The new instance ψ has variables $x_1, \dots, x_n, y_1, \dots, y_m$, where $y_i \in \{0, 1\}^{q_0}$ codes up an assignment to $x_1, \dots, x_n$.
- ▶ For each variable x_j in φ_i , construct the constraint ψ_{ij} stating that y_i satisfies φ_i and y_i is consistent to x_j .

What is the formula $\psi_{ij}(y_i, x_j)$? What is the size of $\psi_{ij}(y_i, x_j)$?

Step 1: Reduction to Nice Instance

Step 1.2. There exist an **absolute** constant d and a CL-reduction that maps a 2CSP_W instance φ to a 2CSP_W instance ψ such that

$$\text{val}(\varphi) \leq 1 - \epsilon \Rightarrow \text{val}(\psi) \leq 1 - \frac{\epsilon}{100Wd}$$

and that G_ψ is **d -regular**.

Let $\{G_k\}_k$ be an explicit $(d-1, 0.9)$ -expander. We get ψ by replacing each k -degree node of G_φ by G_k and adding the identity constraint (of the form $y_i^j = y_i^{j'}$, where $i \in [n]$) to each edge (j, j') of G_k . If φ has m constraints, ψ has dm constraints.

Suppose $\text{val}(\varphi) \leq 1 - \epsilon$ and $\mathbf{v}$ is an unsatisfying assignment to ψ .

It suffices to prove that $\mathbf{v}$ violates at least $\frac{\epsilon m}{100W}$ constraints of ψ .

Continue on the next slide.

Fact. For every $c \in (0, 1)$ there is a constant d and an algorithm that, given input n , runs in $\text{poly}(n)$ time and outputs an (n, d, c) -expander.

Step 1: Reduction to Nice Instance

Let $\mathbf{u}$ be the assignment to φ that is defined by the plurality of the assignment $\mathbf{v}$ to ψ .

Let t_i be the number of v_i^j 's, where $j \in [k]$, that disagree with u_i . Clearly $t_i \leq k(1 - \frac{1}{W})$.

If $\sum_{i=1}^n t_i$ is large, each G_k already contains enough violated constraints.

1. $\sum_{i=1}^n t_i \geq \frac{1}{4}\epsilon m$. Let $S_i = \{y_i^j \mid v_i^j = u_i\}$ and let $\overline{S}_i = \{y_i^1, \dots, y_i^k\} \setminus S_i$. The number of constraints of G_k violated by $\mathbf{v}$ is at least

$$E(S_i, \overline{S}_i) \stackrel{(1)}{\geq} (1 - \lambda_{G_k}) \frac{(d-1)|S_i||\overline{S}_i|}{|S_i| + |\overline{S}_i|} = \frac{1}{10} \frac{d-1}{k} |S_i||\overline{S}_i| \geq \frac{1}{10W} t_i,$$

where $\geq$ is due to $|S_i| \geq k/W$. Now $\sum_{i \in [n]} E(S_i, \overline{S}_i) \geq \frac{\epsilon m}{40W} = \frac{\epsilon}{40Wd} \cdot dm$.

2. $\sum_{i=1}^n t_i < \frac{1}{4}\epsilon m$. Since $\text{val}(\varphi) \leq 1 - \epsilon$, there is at least ϵm constraints violated in φ by $\mathbf{u}$. These ϵm constraints are also in ψ with variables being $\mathbf{v}$.

Since every constraint has two variables, less than $\frac{1}{4}\epsilon m + \frac{1}{4}\epsilon m$ constraints have valuations in ψ different from those in φ . So at least $\frac{1}{2}\epsilon m$ constraints are violated.

Step 1: Reduction to Nice Instance

Step 1.3. There is an **absolute** constant d and a CL-reduction that maps a 2CSP_W instance φ with G_φ being d' -regular for some $d' \leq d$ to a 2CSP_W instance ψ such that

$$\text{val}(\varphi) \leq 1 - \epsilon \Rightarrow \text{val}(\psi) < 1 - \frac{\epsilon}{10d}$$

and that G_ψ is nice, $4d$ -regular, and half of the edges adjacent to each vertex are loops.

There is a constant d and an explicit $(d, 0.1)$ -expander $\{G_n\}_{n \in \mathbb{N}}$. We may assume that φ is d -regular (**adding** self-loops if $d' < d$) and that φ contains n variables.

We get ψ from φ by **adding** a tautological constraint for every edge of G_n and **adding** $2d$ tautological constraints forming self-loops for each vertex. Then

$$\lambda_{G_\psi} \leq \frac{3}{4} + \frac{1}{4}\lambda_{G_\varphi} < 0.9.$$

Notice that “**adding**” decreases ϵ by a factor $\leq d$ and “**adding**” by a further factor ≤ 4 .

Step 2: Gap Amplification

To amplify the gap, we apply a path-product like operation on constraint graphs so that a new constraint is a conjunction of some old constraints.

Path construction increases the fraction of violated constraints.

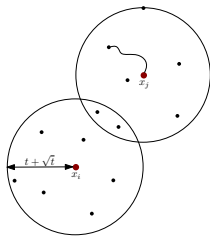
We need to address the issue of how to compose two-variable constraints so that every new constraint contains two variables.

Step 2: Gap Amplification

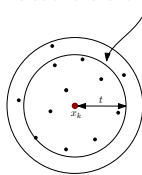
Construction of the $2\text{CSP}_{W'}$ instance ψ^t : **Variables**.

1. Let $x_1, \dots, x_n$ denote the variables of ψ .
2. ψ^t contains n variables $\mathbf{y}_1, \dots, \mathbf{y}_n \in W' < W^{d^{5t}}$, where $\mathbf{y}_i$ is an assignment to those of $x_1, \dots, x_n$ reachable within $t + \delta(t)$ steps from x_i . We will let $\delta(t) = \sqrt{t}$.

For $i, j \in [n]$ we say that an assignment to $\mathbf{y}_i$ **claims** a value for x_j .



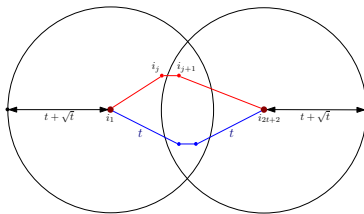
The belt zone is for consistence checking.



Step 2: Gap Amplification

Construction of the $2\text{CSP}_{W'}$ instance ψ^t : **Constraints.**

1. For each $2t+1$ step path $p = (i_1, \dots, i_{2t+2})$ in G_ψ , introduce $C_p = \bigwedge_{j \in [2t+2]} C_p^j$ as a constraint of ψ^t such that for each j that C_p^j appears in C_p , the following hold:
 - 1.1 The length of $(i_1, \dots, i_j)$ and the length of $(i_{j+1}, \dots, i_{2t+2})$ are $\leq t + \sqrt{t}$;
 - 1.2 C_p^j is obtained from the constraint of x_{i_j} and $x_{i_{j+1}}$ by replacing $x_{i_j}, x_{i_{j+1}}$ respectively by $\mathbf{y}_{i_1}$'s claim for x_{i_j} and $\mathbf{y}_{i_{2t+2}}$'s claim for $x_{i_{j+1}}$.
-



Step 2: Gap Amplification

Lemma. There is an algorithm that given $t > 1$ and a nice 2CSP_W instance ψ with n variables, $m = \frac{dn}{2}$ edges, d -degree G_ψ , produces a $2\text{CSP}_{W'}$ instance ψ^t satisfying 1-4.

1. $W' < W^{d^{5t}}$ and ψ^t has at most $n \cdot d^{t+\sqrt{t}+1}$ constraints.
2. If ψ is satisfiable then ψ^t is satisfiable.
3. For $\epsilon < \frac{1}{d\sqrt{t}}$, if $\text{val}(\psi) \leq 1 - \epsilon$, then $\text{val}(\psi^t) \leq 1 - \ell\epsilon$ for $\ell = \frac{\sqrt{t}}{10^4 d W^5}$.
4. The formula ψ^t is produced in $\text{poly}(m, W^{d^{5t}})$ time.

Gap Amplification follows immediately from the lemma.

- ▶ If $\ell = 6$ and $W = 2^{q_0}$, we get a constant t and a constant $\epsilon_0 = \frac{1}{d\sqrt{t}}$.
- ▶ In this case ψ^t has $O(m)$ constraints and is produced in $\text{poly}(m)$ time.

Step 2: Gap Amplification

The conditions 1, 2, 4 of the lemma are satisfied.

Step 2: Gap Amplification

Fix an arbitrary assignment $\mathbf{v}_1, \dots, \mathbf{v}_n$ to $\mathbf{y}_1, \dots, \mathbf{y}_n$.

We would like to define an assignment to $x_1, \dots, x_n$ from $\mathbf{v}_1, \dots, \mathbf{v}_n$.

1. Let $Z_i \in [W]$ be a random variable defined by the following.
 - ▶ Starting from the vertex i , take a t step random walk in G_ψ to reach some vertex k ; output $\mathbf{v}_k$'s claim for x_i .

Let w_i denote the most likely value of Z_i .

2. We call $w_1, \dots, w_n$ the **plurality assignment** to $x_1, \dots, x_n$. Clearly $\Pr[Z_i = w_i] \geq \frac{1}{W}$.
-

You are what your friends think you are.

Step 2: Gap Amplification

A **random** $2t+1$ step path $p = (i_1, \dots, i_{2t+2})$ is picked up with probability $\frac{2}{n \cdot d^{2t+1}}$.

Equivalently such a random path can be chosen in either of the following manners.

1. Pick up a random node, and take a $2t+1$ step random walk from the node.
2. Pick up a random edge, and then take a j -step random walk from one node of the edge and a $(2t-j)$ -step random walk from the other node of the edge.

Pick up a random constraint of ψ^t is the same as picking up a random $2t+1$ step path.

Step 2: Gap Amplification

For $j \in [2t+1]$ the j -th edge (i_j, i_{j+1}) in p is **truthful** if $\mathbf{v}_{i_1}$ claims the plurality value for i_j and $\mathbf{v}_{i_{2t+2}}$ claims the plurality value for i_{j+1} .

Suppose $\text{val}(\psi) \leq 1 - \epsilon$. There is a set F of $\epsilon m = \epsilon \frac{dn}{2}$ constraints in ψ violated by the assignment $x_1 = w_1, \dots, x_n = w_n$

- If p has an edge that is both **truthful** and in F , the constraint C_p is violated.
-

Claim. Let $\delta = \frac{1}{100W}$. For each $j \in [t+1, t + \delta\sqrt{t}]$,

the proof is on the next slide

$$\Pr_p \Pr_{e \in_R E} [\text{the } j\text{th edge of } p \text{ is truthful} \mid \text{the } j\text{th edge of } p \text{ is } e] \geq \frac{1}{2W^2}.$$

Consequently an edge in p is violated with probability at least $\frac{\epsilon}{2W^2}$.

A $2t+1$ step random walk with $(i_j, i_{j+1}) = e$ can be generated by a $(j-1)$ -step random walk from i_j and a $(2t-j+1)$ -step random walk from i_{j+1} . It boils down to proving

$$\Pr[\mathbf{v}_{i_1} \text{ claims the plurality value for } i_j] \cdot \Pr[\mathbf{v}_{i_{2t+2}} \text{ claims the plurality value for } i_{j+1}] \geq \frac{1}{2W^2}. \quad (4)$$

If $j = t + 1$ then the left hand side of (4) is at least $1/W^2$. Otherwise observe that

- ▶ a j -step random walk can be generated by tossing coin for j times and taking S_j -step non-self-loop random walk, where $S_j = \# \text{head's}$, and that
- ▶ the statistical distance $\Delta(S_t, S_{t+\delta\sqrt{t}})$ is bounded by 10δ .

Intuitively it is very likely that starting from a same vertex a $(t+\delta\sqrt{t})$ -step random walk and a t -step walk would end up in the same vertex.

Thus the left hand side of (4) is at least $(\frac{1}{W} - 10\delta) (\frac{1}{W} - 10\delta) \geq \frac{1}{2W^2}$.

Step 2: Gap Amplification

Let V be the random variable for the number of edges among the middle $\delta\sqrt{t}$ edges that are **truthful** and in F .

$\Pr[V > 0]$ is the probability of a ψ^t 's constraint being violated. If $\Pr[V > 0] \geq \epsilon'$, then at least ϵ' fraction of ψ^t 's constraints are violated.

Lemma. For every non-negative random variable V , $\Pr[V > 0] \geq \frac{E[V]^2}{E[V^2]}$.

Proof.

$E[V|V>0]^2 \leq E[V^2|V>0]$ by convex property. The lemma follows from the following.

- ▶ $E[V^2|V>0] = \sum_i i^2 \cdot \Pr[V=i|V>0] = \sum_i i^2 \cdot \frac{\Pr[V=i]}{\Pr[V>0]} = \frac{E[V^2]}{\Pr[V>0]}.$
- ▶ $E[V|V>0]^2 = (\sum_i i \cdot \Pr[V=i|V>0])^2 = \left(\sum_i i \cdot \frac{\Pr[V=i]}{\Pr[V>0]}\right)^2 = \left(\frac{E[V]}{\Pr[V>0]}\right)^2.$



Step 2: Gap Amplification

Claim. $E[V] \geq \frac{\delta\sqrt{t}\epsilon}{2W^2}$.

Proof.

By the previous claim, the probability of an edge in the middle interval of size $\delta\sqrt{t}$ that is truthful and in F is at least $\frac{\epsilon}{2W^2}$. Then $E[V] \geq \frac{\delta\sqrt{t}\epsilon}{2W^2}$ by linearity. $\square$

Claim. $E[V^2] \leq 11\delta\sqrt{t}d\epsilon$.

Proof.

- ▶ Let V' be the number of edges in the middle interval that are in F . Now $V \leq V'$. It suffices to show that $E[V'^2] \leq 11\delta\sqrt{t}d\epsilon$.
- ▶ For $j \in \{t+1, \dots, t+\delta\sqrt{t}\}$, let I_j be an indicator random variable that is 1 if the j th edge is in F and 0 otherwise. Then $V' = \sum_{j \in \{t+1, \dots, t+\delta\sqrt{t}\}} I_j$.
- ▶ Let S be the set of end points of the edges in F . Then $\frac{|S|}{dn} \leq \epsilon$. continue on next slide.

Step 2: Gap Amplification

$$\begin{aligned}
 \mathbb{E}[V'^2] &= \mathbb{E}\left[\sum_j l_j^2\right] + \mathbb{E}\left[\sum_{j \neq j'} l_j l_{j'}\right] \\
 &= \epsilon \delta \sqrt{t} + 2 \sum_{j < j'} \Pr[j\text{th edge is in } F \wedge j'\text{th edge is in } F] \\
 &\leq \epsilon \delta \sqrt{t} + 2 \sum_{j < j'} \Pr[j\text{th vertex of walk lies in } S \wedge j'\text{th vertex of walk lies in } S] \\
 &\leq \epsilon \delta \sqrt{t} + 2 \sum_j \Pr[j\text{th vertex of walk lies in } S] \cdot \sum_{j' > j} \Pr_{(a,b) \in E^{j'-j}}[a \in S, b \in S] \\
 &\stackrel{(3)}{\leq} \epsilon \delta \sqrt{t} + 2 \sum_j d\epsilon \cdot \sum_{j' > j} d\epsilon \left(\frac{1}{2} + \frac{(\lambda_G)^{j'-j}}{2} \right) \\
 &\leq \epsilon \delta \sqrt{t} + (\delta \sqrt{t})^2 (d\epsilon)^2 \left(1 + \sum_{k \geq 1} (\lambda_G)^k \right) \\
 &\leq 11 \delta \sqrt{t} d \epsilon.
 \end{aligned}$$

$\lambda_G < 0.9$ and $\delta < 1$ and $\sqrt{t} d \epsilon < 1$

Step 2: Gap Amplification

Finally we conclude that

$$\begin{aligned}\Pr[V > 0] &\geq \frac{\mathbb{E}[V]^2}{\mathbb{E}[V^2]} \\ &\geq \left(\frac{\delta\sqrt{t}\epsilon}{2W^2}\right)^2 \cdot \frac{1}{11\delta\sqrt{t}d\epsilon} \\ &= \delta \cdot \frac{\sqrt{t}}{44dW^4} \cdot \epsilon \\ &= \frac{1}{100W} \cdot \frac{\sqrt{t}}{44dW^4} \cdot \epsilon \\ &> \frac{\sqrt{t}}{10^4dW^5} \epsilon \\ &= \ell\epsilon.\end{aligned}$$

Step 3: Alphabet Reduction

We look for an algorithm that transforms a 2CSP_W to a $q_0\text{CSP}$ instance.

A simple idea is to turn a variable over $[W]$ to $\log W$ boolean variables.

- ▶ A constraint can be turned into a circuit C of size bounded by $2^{2\log W} < W^4$.
 - ▶ This would produce a CSP instance of arity $2\log W$.
-

The problem with this idea is that $2\log W$ is greater than q_0 (in fact $W \geq 2^{q_0}$).

- ▶ If we apply **Gap Amplification** and **Alphabet Reduction** for $\log m$ times, we would get a CSP instance whose arity depends on input size.

Step 3: Alphabet Reduction

A more sophisticated idea is to design a PCP checker for constraint checking!

1. We turn the $2CSP_W$ problem to evaluation checking problem for CKT-SAT.
 2. We further turn it to solution checking problem for QUADEQ.
 3. We then apply the construction of the PCP verifier (with q_0 queries!) for QUADEQ.
 4. Finally we turn the PCP verifier to a q_0 CSP instance.
-

PCP of Proximity, Verifier Composition, Proof Composition.

- ▶ In some occasions a verifier is allowed to make only a small or constant number of queries. In other words it cannot read any complete assignment to variables.
- ▶ A solution is to see an assignment as part of a proof. Consequently a verifier can only get to see a fragment of the proof.

This is the PCP verifier for QUADEQ!

Step 3: Alphabet Reduction

Suppose a constraint has been converted to a QUADEQ instance.

- ▶ Let $\mathbf{u}_1$ and $\mathbf{u}_2$ be assignments to $\log W$ variables.
- ▶ Let $\mathbf{c}$ be bit string of size $\ell = \text{poly}(W)$ that represents the quadratic equations derived from the circuit C . We assume that the first $2 \log W$ bits of $\mathbf{c}$ are $\mathbf{u}_1\mathbf{u}_2$.

Let $\pi_1\pi_2\pi_3$ be a PCP proof for the QUADEQ instance, where

- ▶ π_1 is supposedly $\text{WH}(\mathbf{u}_1)$, π_2 is supposedly $\text{WH}(\mathbf{u}_2)$ and π_3 is supposedly $\text{WH}(\mathbf{c})$.

Step 3: Alphabet Reduction

The PCP verifier does the following:

1. Check that π_1 , π_2 and π_3 are 0.99-close to $\text{WH}(\mathbf{u}_1)$, $\text{WH}(\mathbf{u}_2)$ and $\text{WH}(\mathbf{c})$ respectively.
2. Check that the first $2 \log W$ bits of $\mathbf{c}$ are $\mathbf{u}_1 \mathbf{u}_2$. This is done by **concatenation test**:
 - 2.1 Choose randomly $\mathbf{x}, \mathbf{y} \in \{0, 1\}^{\log W}$.
 - 2.2 Check that $\pi_3(\mathbf{x} \mathbf{y} 0^{|\mathbf{c}| - 2 \log W}) = \pi_1(\mathbf{x}) + \pi_2(\mathbf{y})$.

Confer the reduction from CKT-SAT to QUADEQ.

Step 3: Alphabet Reduction

PCP of Proximity.

There is a PCP verifier for every old constraint.

There is a verifier $\mathbb{V}$ that, given any circuit C with $2k$ input variables, runs in $\text{poly}(|C|)$ time, uses $\text{poly}(|C|)$ random bits, and enjoys the following property.

1. If $\mathbf{u}_1, \mathbf{u}_2 \in \{0, 1\}^k$ and $\mathbf{u}_1\mathbf{u}_2$ is a satisfying assignment for C , then there is some $\pi_3 \in \{0, 1\}^{2^{\text{poly}(|C|)}}$ such that $\mathbb{V}$ accepts $\text{WH}(\mathbf{u}_1)\text{WH}(\mathbf{u}_2)\pi_3$ with probability 1.
2. For $\pi_1, \pi_2 \in \{0, 1\}^{2^k}$ and $\pi_3 \in \{0, 1\}^{2^{\text{poly}(|C|)}}$, if $\mathbb{V}$ accepts $\pi_1\pi_2\pi_3$ with probability $> 1/2$, then π_1 and π_2 are 0.99-close to $\text{WH}(\mathbf{u}_1)$ and $\text{WH}(\mathbf{u}_2)$ respectively for some $\mathbf{u}_1, \mathbf{u}_2 \in \{0, 1\}^k$, where $\mathbf{u}_1\mathbf{u}_2$ is a satisfying assignment to C .

Step 3: Alphabet Reduction

The PCP verifier can be turned into a q_0 CSP instance. This is the proof of the equivalence between PCP Theorem and PCP Theorem.

Step 3: Alphabet Reduction

Fact. If the value of the old CSP is $\leq 1-\epsilon$, then the value of the new CSP is $\leq 1-\frac{1}{3}\epsilon$.

Suppose an assignment to the new variables satisfied $> 1-\frac{1}{3}\epsilon$ fraction of the new constraints. By decoding an assignment to the old variables satisfied a $1-\delta$ fraction of the old constraints. For each violated old constraint C_s , at least half of the set C_s of the new constraints is violated. Thus $\frac{1}{2}\delta \leq \frac{1}{3}\epsilon$. So at least $1-\delta \geq 1-\frac{2}{3}\epsilon > 1-\epsilon$ fraction of the old constraints were violated, contradicting to the assumption.

This finishes the proof of **Alphabet Reduction**.

Threshold Result by Håstad's 3-Bit PCP Theorem

Håstad Theorem. For every $\delta \in (0, 1)$ and every $L \in \mathbf{NP}$, there is a PCP verifier $\mathbb{V}$ for L that makes three binary queries and satisfies completeness with parameter $1 - \delta$ and soundness with a parameter at most $\frac{1}{2} + \delta$. Moreover, given a proof $\pi \in \{0, 1\}^m$, $\mathbb{V}$ chooses $i_1, i_2, i_3 \in [m]$ and $b \in \{0, 1\}$ according to some distribution and accepts iff $\pi_{i_1} + \pi_{i_2} + \pi_{i_3} = b \pmod{2}$.

1. J. Håstad. Some Optimal Inapproximability Results. J. ACM, 48:798-859, 2001, also in STOC'97.

Threshold Result

An instance of **MAX-E3LIN** consists of finitely many equations of the form

$$x_{i_1} + x_{i_2} + x_{i_3} = b \quad (5)$$

taking values in $GF(2)$. One looks for the size of the largest set of satisfiable equations.

According to Håstad Theorem an NP-complete problem has a PCP verifier that checks if a proof validates equations of the form (5). It is NP-hard to compute a $\frac{1/2+\delta}{1-\delta}$ -approximation to MAX-E3LIN. In other words $(\frac{1}{2}+\epsilon)$ -approximation to MAX-E3LIN is NP-hard for all $\epsilon \in (0, 1/2)$.

It is straightforward to give a $\frac{1}{2}$ -approximation algorithm for MAX-E3LIN.

Threshold Result for MAX-3SAT

Fact. For all $\epsilon \in (0, 1/8)$ computing $(\frac{7}{8} + \epsilon)$ -approximation to MAX-3SAT is NP-hard.

Convert $x + y + z = 0$ to four clauses $\bar{x} \vee y \vee z$, $x \vee \bar{y} \vee z$, $x \vee y \vee \bar{z}$, $\bar{x} \vee \bar{y} \vee \bar{z}$ and $x + y + z = 1$ to four clauses $x \vee \bar{y} \vee \bar{z}$, $\bar{x} \vee y \vee \bar{z}$, $\bar{x} \vee \bar{y} \vee z$, $x \vee y \vee z$.

If an assignment to x, y, z satisfies the equation, it satisfies all the four clauses.
Otherwise it satisfies three of the four clauses.

With the help of the previous reduction, we see that in the yes case at least a $1 - \delta$ fraction of the clauses are satisfied, and in the no case at most a $1 - (\frac{1}{2} - \epsilon) \times \frac{1}{4}$ fraction of the clauses are satisfied.

There is a $\frac{7}{8}$ -approximation algorithm to MAX-3SAT.

Proof of Håstad's 3-Bit PCP Theorem

Long Code

Let $W \in \mathbb{N}$.

A function $f : \{\pm 1\}^W \rightarrow \{\pm 1\}$ is a **coordinate function** if $f(x_1, \dots, x_W) = x_w$ for some $w \in W$. In other words $f = \chi_{\{w\}}$.

The **long code** for $[W]$ encodes each $w \in [W]$ by all the values of $\chi_{\{w\}}$.

Local Test for Long Code

Let $\delta \in (0, 1)$.

1. Choose $\mathbf{x}, \mathbf{y} \in_{\mathcal{R}} \{\pm 1\}^W$.
 2. Choose a noise vector $\mathbf{z} \in_{\mathcal{R}} \{\pm 1\}^W$ using following distribution: For $i \in [W]$, choose $z_i = +1$ with probability $1 - \rho$ and $z_i = -1$ with probability ρ .
 3. Accept if $f(\mathbf{xy}) = f(\mathbf{xyz})$, reject otherwise.
-

If $f = \chi_{\{w\}}$, then $f(\mathbf{x})f(\mathbf{y})f(\mathbf{xyz}) = x_w y_w (x_w y_w z_w) = z_w$. The test accepts iff $z_w = 1$, which happens with probability $1 - \rho$.

Long Code

Lemma. If the test accepts with probability $\frac{1}{2} + \delta$, then $\sum_{\alpha} \hat{f}_{\alpha}^3 (1 - 2\rho)^{|\alpha|} \geq 2\delta$.

If the test accepts with probability $\frac{1}{2} + \delta$, then $\mathbb{E}_{\mathbf{x}, \mathbf{y}, \mathbf{z}}[f(\mathbf{x})f(\mathbf{y})f(\mathbf{xyz})] = 2\delta$. By Fourier expansion,

$$\begin{aligned} 2\delta &\leq \mathbb{E}_{\mathbf{x}, \mathbf{y}, \mathbf{z}} \left[\left(\sum_{\alpha} \hat{f}_{\alpha} \chi_{\alpha}(\mathbf{x}) \right) \left(\sum_{\beta} \hat{f}_{\beta} \chi_{\beta}(\mathbf{y}) \right) \left(\sum_{\gamma} \hat{f}_{\gamma} \chi_{\gamma}(\mathbf{xyz}) \right) \right] \\ &= \mathbb{E}_{\mathbf{x}, \mathbf{y}, \mathbf{z}} \left[\sum_{\alpha, \beta, \gamma} \hat{f}_{\alpha} \hat{f}_{\beta} \hat{f}_{\gamma} \chi_{\alpha}(\mathbf{x}) \chi_{\beta}(\mathbf{y}) \chi_{\gamma}(\mathbf{x}) \chi_{\gamma}(\mathbf{y}) \chi_{\gamma}(\mathbf{z}) \right] = \sum_{\alpha} \hat{f}_{\alpha}^3 \mathbb{E}_{\mathbf{x}}[\chi_{\alpha}(\mathbf{z})] \\ &= \sum_{\alpha} \hat{f}_{\alpha}^3 \mathbb{E}_{\mathbf{x}} \left[\prod_{w \in \alpha} z_w \right] = \sum_{\alpha} \hat{f}_{\alpha}^3 \prod_{w \in \alpha} \mathbb{E}_{\mathbf{x}}[z_w] = \sum_{\alpha} \hat{f}_{\alpha}^3 (1 - 2\rho)^{|\alpha|}. \end{aligned}$$

The smaller α is, the more significant $\hat{f}_{\alpha}$ is. See the next corollary.

Corollary. If f passes the long code test with probability $\frac{1}{2} + \delta$, then for $k = \frac{1}{2\rho} \log \frac{1}{\epsilon}$, there is α with $|\alpha| \leq k$ such that $\hat{f}_\alpha \geq 2\delta - \epsilon$.

Historical Remark

Interactive proof, Zero knowledge, **IP**.

It all started with the introduction of interactive proof systems.

1. Shafi Goldwasser, Silvio Micali, Charles Rackoff. The Knowledge Complexity of Interactive Proofs. STOC'85, SIAM, 1989.
2. László Babai and Shlomo Moran. Arthur-Merlin Games: A Randomized Proof System, and a Hierarchy of Complexity Classes. STOC'85, JCSS, 1988.

The authors of the papers shared the first Gödel Prize (1993).

Goldwasser and Sipser. Private Coins versus Public Coins in Interactive Proof Systems. STOC'86.

“1989 was an extraordinary year.”

László Babai, 1990

1. N. Nisan. Co-SAT has multi-prover interactive proofs, e-mail announcement. Nov. 27, 1989.
2. C. Lund, L. Fortnow, H. Karloff, N. Nisan. The polynomial time hierarchy has interactive proofs, e-mail announcement, Dec. 13, 1989.
3. A. Shamir. $IP=PSPACE$, e-mail announcement, Dec. 26, 1989.

On Jan. 17, 1990 another email was sent out by L. Babai, L. Fortnow, and L. Lund.

- ▶ Non-Deterministic Exponential Time has Two Prover Interactive Protocols. FOCS 1990. CC 1991.

The main theorem of the paper, **MIP** = **NEXP**, inspired almost all future development of PCP theory and a lot of future development in derandomization theory. It can be interpreted as

$$\text{NEXP} = \text{PCP}(\text{poly}, \text{poly}).$$

A profitable shift of emphasis was made that, instead of scaling down the time or space complexity of verifier, scales down the randomness and query complexity.

Babai, Fortnow, Levin, and Szegedy showed $\mathbf{NP} \subseteq \mathbf{PCP}(\text{polylog}, \text{polylog})$.

-
1. L. Babai, L. Fortnow, L. Levin, and M. Szegedy. Checking Computation in Polylogarithmic Time. STOC, 1991.

1. $\mathbf{NP} \subseteq \mathbf{PCP}(\log \cdot \log \log, \log \cdot \log \log)$.
2. $\mathbf{NP} = \mathbf{PCP}(\log, \log)$.
3. $\mathbf{NP} = \mathbf{PCP}(\log, 1)$.



1. U. Feige, S. Goldwasser, L. Lovász, S. Safra, and M. Szegedy. Interactive Proofs and the Hardness of Approximating Cliques. FOCS'91, JACM, 1996.
2. S. Arora and S. Safra. Probabilistic Checking of Proofs: A New Characterization of NP. FOCS'92, JACM, 1998.
3. S. Arora, C. Lund, R. Motwani, M. Sudan, and M. Szegedy. Proof Verification and the Hardness of Approximation Problems. FOCS'92, JACM, 1998.

2019 Gödel Prize

Irit Dinur. The PCP Theorem by Gap Amplification. J. ACM, 2007. STOC 2006.

